CP020003

ARTIFICIAL INTELLIGENCE

Week 02

Feature Engineering

(Thinking and Coding Like a Spotify Data Engineer)

Instructor: Teerapong Panboonyuen (P'Kao)

https://github.com/kaopanboonyuen/CP020003_ArtificialIntelligence_2026s1

Khon Kaen University · 2026

ROADMAP

What We'll Build Today

01

Warm-up

Load Spotify data. Recap def / lambda / if-else — the tools of feature engineering.

02

Classic FE

Numeric transforms, ratios, binning, text features — the hackathon fundamentals.

03

Group Thinking

Aggregation features, target-style encoding, and the data-leakage trap.

04

Domain Knowledge

Music-theory mood quadrants — turning theory into signal.

05

LLM-Era FE

Zero-shot mood labels & sentence embeddings, CPU-friendly with HuggingFace.

06

Ship It

Assemble the final feature table and save it like a real pipeline.

FE 101

Feature Engineering: Basic → Advanced

The same discipline, applied at increasing depth. Most winning solutions climb this whole ladder.

BASIC

- Type conversion (str → int, ms → min)
- Missing value handling
- Simple math: ratios, differences

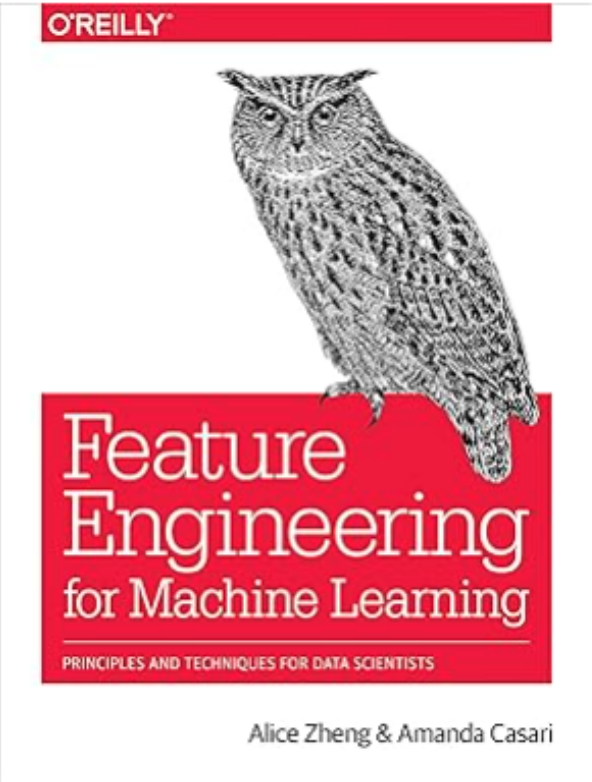
INTERMEDIATE

- Binning & categorical encoding
- Group aggregation / target-style stats
- Scaling & normalization

ADVANCED

- Text & embedding-based features (LLMs)
- Unsupervised signals (PCA, clustering)
- Automated feature selection, feature stores

Books › Computers & Technology › Computer Science › Artificial Intelligence › Machine Learning



Feature Engineering for Machine Learning: Principles and Techniques for Data Scientists

by Alice Zheng (Author), Amanda Casari (Author) | Format: Kindle Edition

4.3 ★★★★☆ (94) 3.8 on Goodreads 156 ratings

See all formats and editions

Feature engineering is a crucial step in the machine-learning pipeline, yet this topic is rarely examined on its own. With this practical book, you'll learn techniques for extracting and transforming features—the numeric representations of raw data—into formats for machine-learning models. Each chapter guides you through a single data problem, such as how to represent text or image data. Together, these examples illustrate the main principles of feature engineering.

Rather than simply teach these principles, authors Alice Zheng and Amanda Casari focus on practical application with exercises throughout the book. The closing chapter brings everything together by tackling a real-world, structured dataset with several feature-engineering techniques. Python packages including numpy, Pandas, Scikit-learn, and Matplotlib are used in code examples.

You'll examine:

- Feature engineering for numeric data: filtering, binning, scaling, log transforms, and power transforms
- Natural text techniques: bag-of-words, n-grams, and phrase detection

Read more

THE BIG IDEA

What is feature engineering?

A working definition

Using domain knowledge and data manipulation to transform raw columns into inputs that better expose the underlying pattern a model needs to learn.

It is NOT just cleaning data — it's actively creating new, more informative variables.

It IS part science (statistics, math) and part craft (intuition, experimentation, iteration).

"Features are the currency of machine learning."

From raw data to model-ready signal

Raw Data

messy, mixed units, missing values

Feature Engineering

transform · combine · encode

Model-Ready Table

clean numeric signal

In Kaggle competitions and real-world hackathons, the winning edge almost always comes from smarter, more creative features — not a marginally better algorithm.

Rich features + simple model

- ✓ Linear/tree model
- ✓ Well-engineered inputs
- ✓ Fast to train & explain
- ✓ Often wins in practice

Raw columns + complex model

- ✗ Deep / ensemble model
- ✗ Unprocessed raw inputs
- ✗ Slow, hard to debug
- ✗ Underperforms surprisingly often

"Garbage in, garbage out."

THE PROCESS

A repeatable feature engineering workflow

Treat it as a loop, not a one-time step — you'll revisit earlier stages as you learn more.

1

Understand

Read the data dictionary, profile columns, talk to domain experts

2

Clean

Handle missing values, fix types, resolve inconsistent units

3

Transform

Scale, encode, bin, parse dates & text

4

Create

Ratios, interactions, aggregations, embeddings

5

Validate

Check leakage, distributions, correlation with target

PRINCIPLE 01

Understand your data before you touch it

The single biggest source of wasted effort is engineering clever features on a column you've misunderstood. Every good feature starts with a genuine read of the data — not just df.head().

Read the data dictionary

Know every column's type, units, and quirks (e.g. "duration" meaning minutes vs. seasons).

Profile distributions

Histograms, value counts, min/max — spot skew, outliers, and encoding surprises early.

Audit missing values

Is a column missing at random, or does missingness itself carry meaning?

Apply domain knowledge

What would a music fan, a Netflix viewer, or a domain expert notice that raw stats won't show?

PRINCIPLE 02

Know your feature-type toolbox

Six families of features cover almost everything you'll build this term.

Numeric transforms

Log, ratios, binning, scaling

Categorical encoding

One-hot, ordinal, frequency, mapping dicts

Text / NLP features

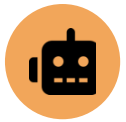
Length, keyword flags, embeddings

Interaction & composite

Combine 2+ columns into a new signal

Aggregation / group stats

groupby().transform(), frequency counts



Unsupervised / embeddings

Clusters, PCA, LLM embeddings

PRINCIPLE 03

Leakage happens when a feature secretly contains information about the target that won't be available at prediction time. Models trained with leakage look brilliant in testing — and fail in production.

⚠ Common leakage traps

- Target encoding without cross-validation folds
- Using future dates (e.g. date_added after the label event)
- Aggregating stats across train + test together
- Features only knowable after the outcome occurs

✓ Safer practice

- Fit encoders / scalers on train data only, then apply to test
- Time-aware splits when data has a timeline
- Ask: "would I know this BEFORE prediction time?"
- Cross-validate any feature that uses the target

PRINCIPLE 04

Handle missing data & outliers thoughtfully

Don't reach for .dropna() by reflex — missingness and extremes are often signal, not noise.

Missing ≠ useless

Add a has_X flag before you decide to impute or drop — the pattern of missingness can be predictive.

Detect outliers first

Z-scores, IQR, or percentile ranks — know which rows are extreme before transforming.

Impute deliberately

Mean/median for numeric, "Unknown" bucket for categorical — always document the choice.

Decide, don't default

Cap, log-transform, or keep outliers — the right call depends on whether they're errors or real extremes.

PRINCIPLE 05

Engineer features like an engineer, not a script

The difference between a notebook hack and a production feature pipeline is reproducibility. The same transformation logic must run identically on train, validation, test — and on tomorrow's new data.

Wrap logic in functions

Not copy-pasted cells — one function per feature, reusable across datasets and splits.

Fit on train, apply everywhere

Scalers, encoders, and imputers are fit once on training data, then applied unchanged elsewhere.

Document every transformation

A future teammate (or future you) should understand why a feature exists, not just how.

Version your feature sets

Track what changed between experiments — reproducible results depend on it.

BEYOND FEATURES

What makes a great data engineer

Feature engineering skill is necessary — but these habits are what make it reliable at scale.

Curiosity

Always asks "what else could this column tell us?"

Rigor & testing

Validates assumptions instead of trusting a hunch

Documentation

Leaves a clear trail for the next person

Automation mindset

Builds pipelines, not one-off notebooks

Ethics & data quality

Watches for bias, leakage, and quiet data drift

Clear communication

Explains *why* a feature matters to non-technical teammates

ANTI-PATTERNS

Common pitfalls to watch for

Leakage from the target

Using information that won't exist at prediction time.

Feature explosion

Adding hundreds of weak features instead of a few strong ones — hurts interpretability and speed.

Silent NaN propagation

Ratios and log-transforms quietly produce NaN/inf that break downstream steps.

Inconsistent train/test logic

Encoding fit on the full dataset instead of train-only — a subtle leakage source.

Ignoring class/category imbalance

A rare category collapses into noise if not aggregated or flagged.

No sanity checks

Never eyeballing a sample of the new feature before moving on.

- Read the data dictionary and profiled every column
- Created features across multiple families (numeric, text, aggregation, embeddings)
- Checked for leakage — nothing uses information from the future or the target unfairly
- Handled missing values and outliers with a deliberate, documented choice
- Wrapped transformations in reusable, reproducible functions

Next week: Week 3 — Supervised Learning

We'll take everything engineered this week and turn it into real predictions — regression and classification models on our feature-rich datasets.

CAREER PREP

What top-tier companies expect at each stage of skill — build this ladder deliberately, one tier at a time.

BASIC

Foundations

- Python & SQL fluency
- Statistics & probability basics
- Git, Linux, command line
- Clean, readable code

Interview focus: coding + fundamentals

INTERMEDIATE

Production Skills

- ETL / ELT pipeline design
- Data warehousing (BigQuery)
- Workflow orchestration (Airflow)
- Data quality & testing

Interview focus: system design lite

ADVANCED

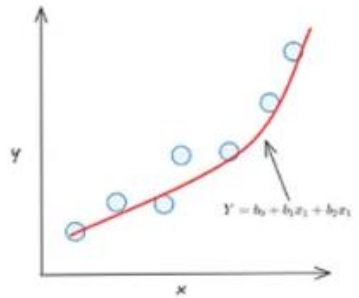
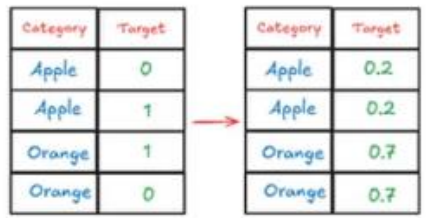
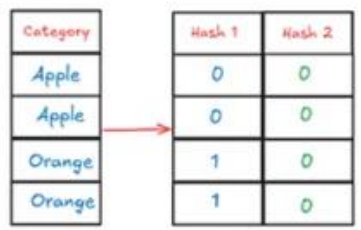
Scale & Systems

- Distributed systems design
- Streaming (Pub/Sub, Dataflow, Kafka)
- Feature stores for ML pipelines
- Reliability at planet-scale

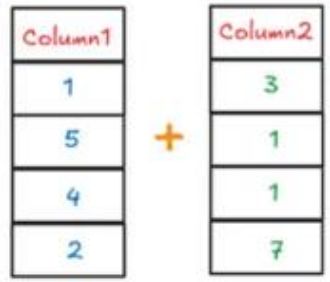
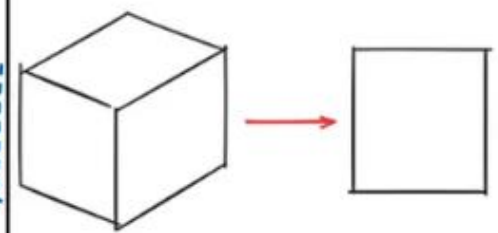
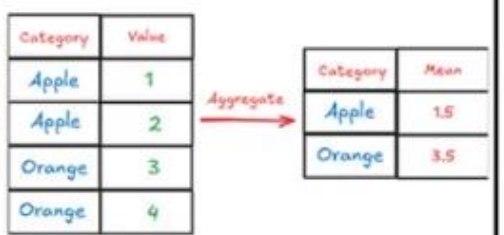
Interview focus: large-scale system design

https://www.geeksforgeeks.org/machine-learning/what-is-feature-engineering/

https://www.nb-data.com/p/8-advance-feature-engineering-for

		Python Implementation	Benefit
Polymonial Features		from sklearn.preprocessing import PolynomialFeatures	Can capture nonlinear relationships.
Target Encoding		from category_encoders import TargetEncoder	Reduces dimensionality for high cardinality
Feature Hashing		from sklearn.feature_extraction import FeatureHasher	Efficient for high-cardinal features
Lag Features		df['lag1'] = df['feature'].shift(1)	Capturing temporal patterns

https://www.nb-data.com/p/8-advance-feature-engineering-for

Feature Interactions		`df['interaction'] = df['feature1'] * df['feature2']`	Improve model performance by capturing feature dependencies
Dimensionality Reduction		`from sklearn.decomposition import PCA`	Improve computational efficiency and reduce overfitting.
Group Aggregation		`df.groupby('category') .agg({'feature': ['mean', 'sum']})`	Improving predictive power using aggregated information

THE BIG IDEA

"Garbage in, garbage out."

No matter how fancy your model is next week, if your features are weak, your model will be weak. This week is 100% about creating great features — the raw material of every winning Kaggle or hackathon solution.

- Model quality is capped by feature quality
- Domain knowledge beats brute force
- A good data engineer builds the ceiling

DATASET

The Spotify KKU Dataset

One row = one track. 20 raw columns describing audio, metadata, and popularity — the raw material we'll turn into features all class.

`track_name / artists / album_name`	Text metadata
`popularity (0-100)`	Target-like signal
`duration_ms`	Numeric, raw
`danceability, energy, valence, tempo...`	Audio-analysis features
`key, mode`	Encoded music theory
`track_genre`	Categorical label

```
RAW_URL = ".../spotify-kku-dataset.csv"

df = pd.read_csv(RAW_URL, index_col=0)
print("Shape:", df.shape)
df.info()

df["track_genre"].value_counts().head(10)
```

👀 **Data engineer habit #1**

Always run .shape, .info(), and .describe() before writing a single feature. You cannot engineer what you don't understand.


Spotify®

1. Load & Inspect the Dataset

We load the CSV directly from GitHub (raw link), so this notebook works out-of-the-box on Colab with zero manual upload.

[3] ✓ 1s

```
RAW_URL = "https://raw.githubusercontent.com/kaopanboonyuen/CP020003_ArtificialIntelligence_2026s1/main/dataset/spotify-kku-dataset.csv"

df = pd.read_csv(RAW_URL, index_col=0)
print("Shape:", df.shape)
df.head()
```

```
Shape: (114000, 20)
```

	track_id	artists	album_name	track_name	popularity	duration_ms	explicit	danceability	energy	key	loudness	mode	speechiness	a
0	5SuOikwiRyPMVoIQDJUgSV	Gen Hoshino	Comedy	Comedy	73	230666	False	0.676	0.4610	1	-6.746	0	0.1430	
1	4qPNDBW1i3p13qLCt0Ki3A	Ben Woodward	Ghost (Acoustic)	Ghost - Acoustic	55	149610	False	0.420	0.1660	1	-17.235	1	0.0763	
2	1iJBSr7s7jYXzM8EGcbK5b	Ingrid Michaelson;ZAYN	To Begin Again	To Begin Again	57	210826	False	0.438	0.3590	0	-9.734	1	0.0557	
3	6lfxq3CG4xtTiEg7opyCyx	Kina Grannis	Crazy Rich Asians (Original Motion Picture Sou...	Can't Help Falling In Love	71	201933	False	0.266	0.0596	0	-18.515	1	0.0363	
4	5vjLSffimiIP26QG5WcN2K	Chord Overstreet	Hold On	Hold On	82	198853	False	0.618	0.4430	2	-9.681	1	0.0526	

RECAP

The Tools Behind Every Feature

Feature engineering is just writing small functions that transform columns into new columns. Three tools, over and over:

def

A normal function. Reusable, testable, can hold if/elif/else and loops.

```
def tempo_category(bpm):
    if bpm < 90:
        return "slow"
    elif bpm < 130:
        return "medium"
    else:
        return "fast"
```

lambda

A tiny anonymous function. Great for one-line transforms inside .apply().

```
df["duration_min"] = (
    df["duration_ms"]
      .apply(lambda x: x / 60000)
)
```

if / elif / else

The heart of rule-based features — most 'smart' categories are just this.

```
for t in [70, 100, 150]:
    print(t, "->",
          tempo_category(t))
# 70 -> slow, 150 -> fast
```

Rule of thumb: simple math -> lambda inside .apply()/.assign(). Multi-branch logic -> def function with if/elif/else, then .apply()/.map().

RECAP

map() vs apply() vs assign()

method	*best for*	*example*
.map()	Element-wise mapping on a Series (dict or function)	`df["mode"].map({0:"minor",1:"major"})`
.apply()	Apply a function row- or element-wise	`df["tempo"].apply(tempo_category)`
.assign()	Chain-create multiple new columns in a pipeline	`df.assign(new=lambda d: d["x"]*2)`

`List comprehension is the fourth tool — a compact loop:  [1 if d > 0.8 else 0 for d in df["danceability"]]`

SECTION 03

Basic Numeric Feature Engineering

Simple unit conversions and rescalings that make raw numbers model-friendly and human-readable.

```
df["duration_min"] = df["duration_ms"] / 60000

# louder = bigger number
df["loudness_abs"] = df["loudness"].abs()

# rescale popularity 0-100 -> 0-1
df["popularity_norm"] = df["popularity"] / 100

df["tempo_rounded"] = df["tempo"].round(0)
```

duration_min

Minutes are easier for humans to reason about than milliseconds.

loudness_abs

Spotify stores loudness as negative dB — abs() flips the intuition to “bigger = louder”.

popularity_norm

0-1 scale plays nicer with many ML algorithms next week.

SECTION 04

Ratio & Interaction Features — “Hackathon Gold”

Combining two or more columns often reveals patterns a single column can't see alone.

dance_energy_score	`danceability * energy`	High only when a song is BOTH danceable and energetic — a party score.
popularity_per_minute	`popularity / duration_min`	Reward for being catchy fast, not just long and popular.
vocal_energy_gap	`speechiness - instrumentalness`	Positive = spoken-word heavy; negative = mostly instrumental.
acoustic_instr_index	`(acousticness + instrumentalness) / 2`	One combined index of a song's 'organic, unplugged' feel.

SECTION 05

Binning & Categorical Encoding

Continuous numbers become buckets; Spotify's numeric key/mode codes become real musical terms — easier for tree models and humans alike.

```
df["duration_bucket"] = pd.cut(
    df["duration_min"], bins=[0, 2.5, 4, 100],
    labels=["short", "medium", "long"])

key_map = {0:"C", 1:"C#/Db", 2:"D", ... 11:"B"}
df["key_name"] = df["key"].map(key_map)

df["mode_name"] = df["mode"].map(
    {0: "minor", 1: "major"})
```

pd.cut vs pd.qcut

pd.cut

Fixed value ranges you choose (e.g. 0-2.5, 2.5-4)

pd.qcut

Equal-sized groups by rank/percentile (quartiles)

key = 0..11 -> C, C#/Db, D … B (12 pitch classes). mode = 0 minor, 1 major. Decoding raw codes into meaning is a core data-engineer skill.

SECTION 06

Text-Based Features

Plain string logic on artists and track_name gives surprisingly strong features — no audio analysis required.

```
df["num_artists"] = df["artists"].apply(
    lambda a: len(str(a).split(";")))

df["is_collab"] = (df["num_artists"] > 1)\
    .astype(int)

df["is_remix"] = df["track_name"].astype(str)\
    .str.contains(r"remix|mix|edit",
                  case=False, regex=True)\
    .astype(int)
```

num_artists — split on ';'

is_collab — 2+ artists?

track_name_length / word_count

is_acoustic_version — regex match

is_remix / is_live

has_parenthesis — feat. tags

Discussion: is_acoustic_version (text) often correlates strongly with acousticness (expensive audio analysis). Can a cheap text feature approximate an expensive one?

SECTION 07

Aggregation Features (Group Statistics)

"How does this row compare to others in the same group?" — one of the most powerful ideas in tabular ML.

```
df["genre_avg_popularity"] = df.groupby(
    "track_genre")["popularity"]\
    .transform("mean")

df["dance_rank_in_genre"] = df.groupby(
    "track_genre")["danceability"]\
    .rank(ascending=False)

df["popularity_z_in_genre"] = (
    df["popularity"] - genre_mean
) / (genre_std + 1e-6)
```

What we built

- Genre average popularity, broadcast to every row
- Danceability rank within genre
- Popularity z-score vs. genre average
- Artist-level average popularity

⚠ Watch for data leakage

Using the target column (popularity) to build group features can leak future information if not done inside cross-validation folds. We build them here to learn the pattern — next week we'll do it safely in a real pipeline.

SECTION 08

Music-Theory-Inspired Features: Mood Quadrant

Inspired by Russell's Circumplex Model of Affect: plotting songs on Valence (positivity) vs Energy.

```
def mood_quadrant(valence, energy):
    if valence >= 0.5 and energy >= 0.5:
        return "Happy"
    elif valence >= 0.5 and energy < 0.5:
        return "Calm"
    elif valence < 0.5 and energy >= 0.5:
        return "Angry"
    else:
        return "Sad"

df["mood_quadrant"] = df.apply(
    lambda r: mood_quadrant(
        r["valence"], r["energy"]), axis=1)
```

SECTION 09

Feature Scaling — A Preview for Week 3

KNN, SVM, logistic regression, and neural nets are sensitive to feature scale. Recognize these two scalers now; we'll use them for modeling next week.

StandardScaler

Rescales to mean = 0, std = 1.
Good for SVM, logistic regression.

```
scaler = StandardScaler()
scaler.fit_transform(
    df[numeric_cols])
```

MinMaxScaler

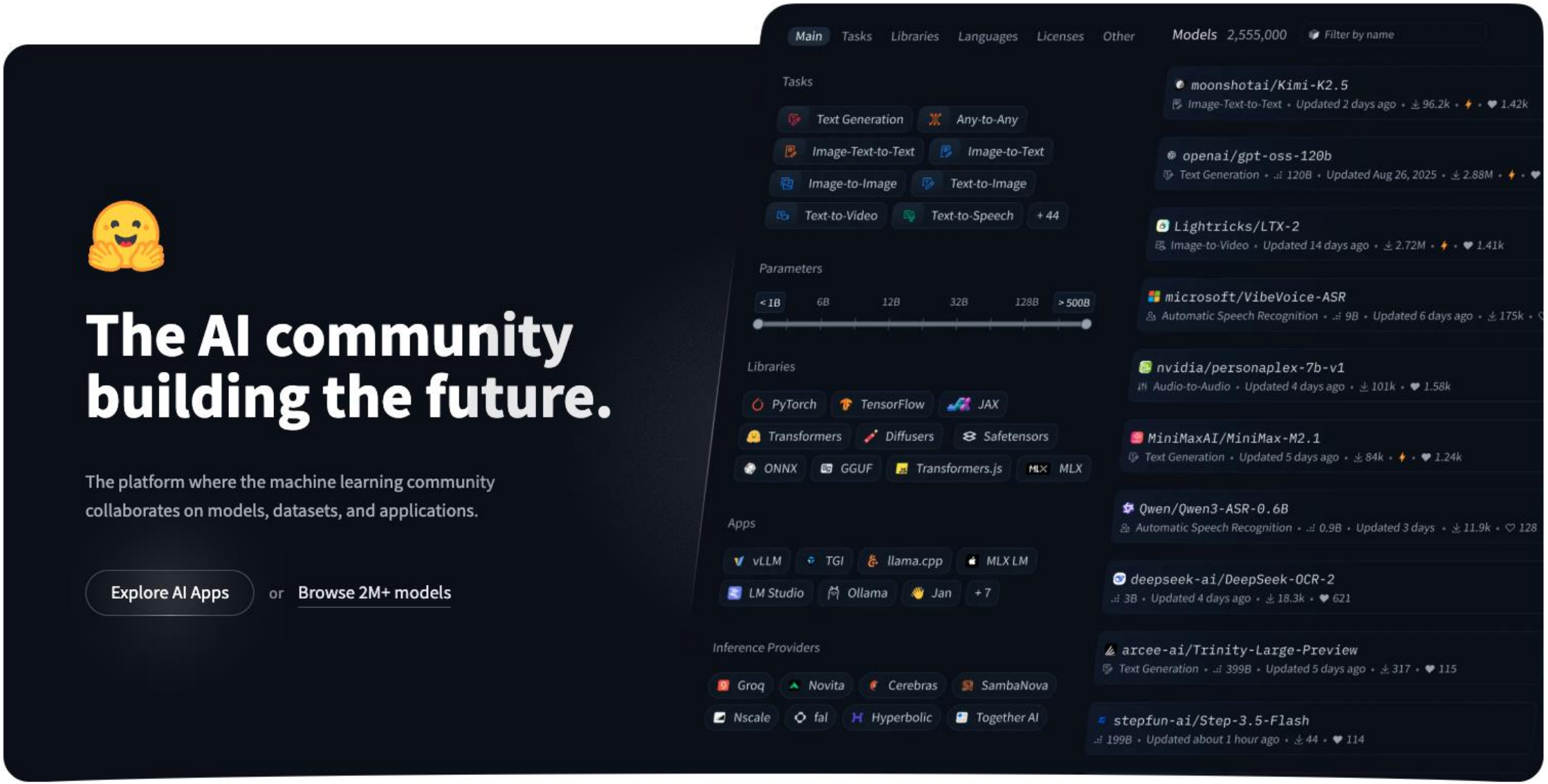
Rescales into a fixed 0-1 range.
Good for neural networks.

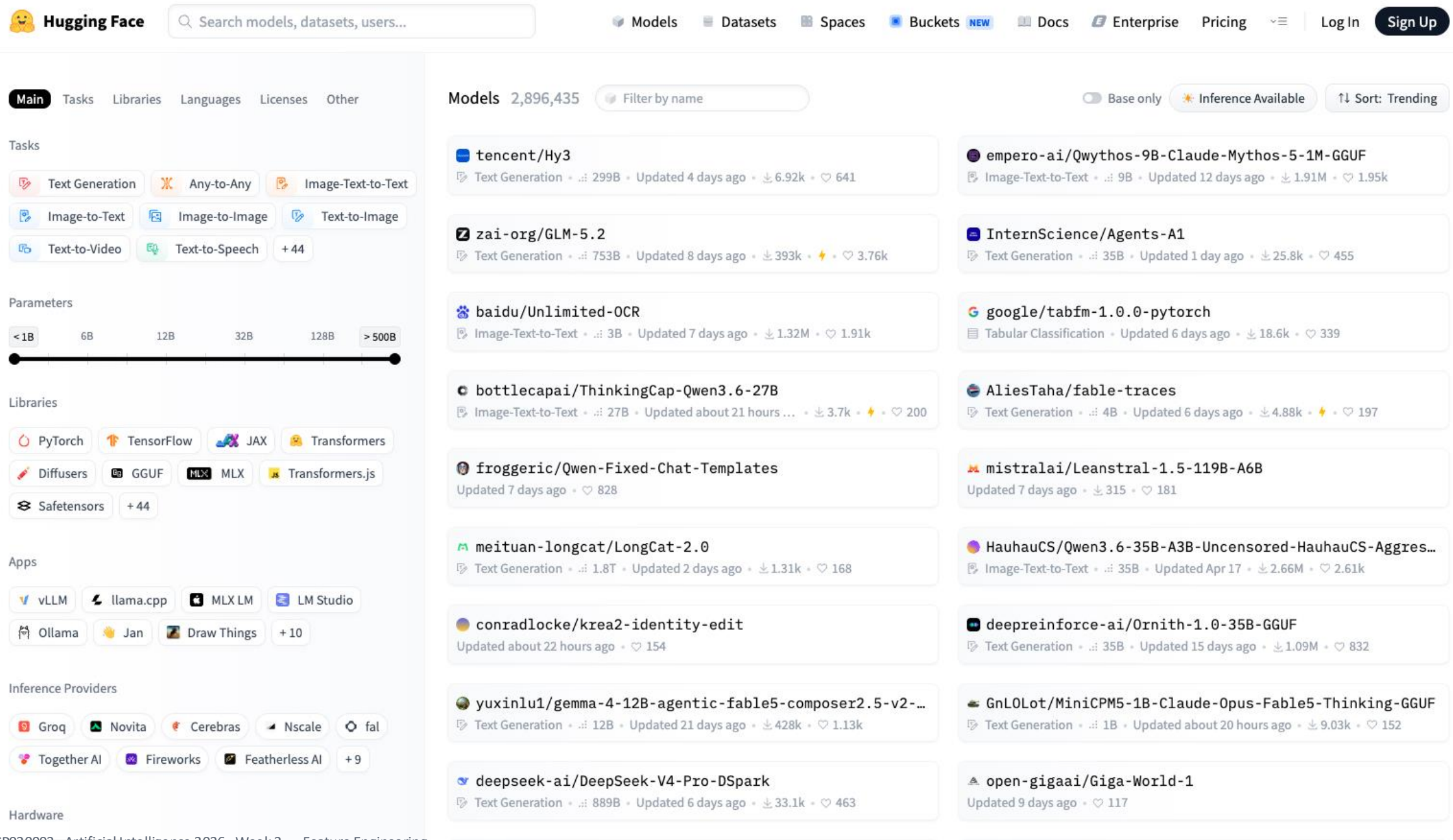
```
scaler = MinMaxScaler()
scaler.fit_transform(
    df[numeric_cols])
```

Data-engineer discipline: fit scalers on training data only, then transform validation/test data with those same parameters — never fit on the full dataset in a real pipeline.

Hugging Face

Hugging Face is essentially the "GitHub of artificial intelligence." It is the world's leading platform and community for machine learning, data science, and open-source AI models.


Hugging Face
Search models, datasets, users...
Models
Datasets
Spaces
Buckets NEW
Docs
Enterprise
Pricing
Log In
Sign Up
The AI community building the future.
The platform where the machine learning community collaborates on models, datasets, and applications.
Explore AI Apps
or
Browse 2M+ models
Main
Tasks
Libraries
Languages
Licenses
Other
Models 2,555,000
Filter by name
Tasks
Text Generation
Any-to-Any
Image-Text-to-Text
Image-to-Text
Image-to-Image
Text-to-Image
Text-to-Video
Text-to-Speech
+ 44
Parameters
< 1B
6B
12B
32B
128B
> 500B
Libraries
PyTorch
TensorFlow
JAX
Transformers
Diffusers
Safetensors
ONNX
GGUF
Transformers.js
MLX
Apps
vLLM
TGI
llama.cpp
MLX LM
LM Studio
Ollama
Jan
+ 7
Inference Providers
Groq
Novita
Cerebras
SambaNova
Nscale
fal
Hyperbolic
Together AI
moonshotai/Kimi-K2.5
Image-Text-to-Text • Updated 2 days ago • 96.2k • 1.42k
openai/gpt-oss-120b
Text Generation • 120B • Updated Aug 26, 2025 • 2.88M
Lightricks/LTX-2
Image-to-Video • Updated 14 days ago • 2.72M • 1.41k
microsoft/VibeVoice-ASR
Automatic Speech Recognition • 9B • Updated 6 days ago • 175k
nvidia/personaplex-7b-v1
Audio-to-Audio • Updated 4 days ago • 101k • 1.58k
MiniMaxAI/MiniMax-M2.1
Text Generation • Updated 5 days ago • 84k • 1.24k
Qwen/Qwen3-ASR-0.6B
Automatic Speech Recognition • 0.9B • Updated 3 days • 11.9k • 128
deepseek-ai/DeepSeek-OCR-2
3B • Updated 4 days ago • 18.3k • 621
arcee-ai/Trinity-Large-Preview
Text Generation • 399B • Updated 5 days ago • 317 • 115
stepfun-ai/Step-3.5-Flash
199B • Updated about 1 hour ago • 44 • 114


Hugging Face
Search models, datasets, users...
Models
Datasets
Spaces
Buckets NEW
Docs
Enterprise
Pricing
Log In
Sign Up
Main
Tasks
Libraries
Languages
Licenses
Other
Tasks
Text Generation
Any-to-Any
Image-Text-to-Text
Image-to-Text
Image-to-Image
Text-to-Image
Text-to-Video
Text-to-Speech
+ 44
Parameters
< 1B
6B
12B
32B
128B
> 500B
Libraries
PyTorch
TensorFlow
JAX
Transformers
Diffusers
GGUF
MLX
Transformers.js
Safetensors
+ 44
Apps
vLLM
llama.cpp
MLX LM
LM Studio
Ollama
Jan
Draw Things
+ 10
Inference Providers
Groq
Novita
Cerebras
Nscale
fal
Together AI
Fireworks
Featherless AI
+ 9
Hardware
Models 2,896,435
Filter by name
Base only
Inference Available
Sort: Trending
tencent/Hy3
Text Generation • 299B • Updated 4 days ago • 6.92k • 641
empero-ai/Qwythos-9B-Claude-Mythos-5-1M-GGUF
Image-Text-to-Text • 9B • Updated 12 days ago • 1.91M • 1.95k
zai-org/GLM-5.2
Text Generation • 753B • Updated 8 days ago • 393k • 3.76k
InternScience/Agents-A1
Text Generation • 35B • Updated 1 day ago • 25.8k • 455
baidu/Unlimited-OCR
Image-Text-to-Text • 3B • Updated 7 days ago • 1.32M • 1.91k
google/tabfm-1.0.0-pytorch
Tabular Classification • Updated 6 days ago • 18.6k • 339
bottlecapai/ThinkingCap-Qwen3.6-27B
Image-Text-to-Text • 27B • Updated about 21 hours ... • 3.7k • 200
AliesTaha/fable-traces
Text Generation • 4B • Updated 6 days ago • 4.88k • 197
froggeric/Qwen-Fixed-Chat-Templates
Updated 7 days ago • 828
mistralai/Leanstral-1.5-119B-A6B
Updated 7 days ago • 315 • 181
meituan-longcat/LongCat-2.0
Text Generation • 1.8T • Updated 2 days ago • 1.31k • 168
HauhauCS/Qwen3.6-35B-A3B-Uncensored-HauhauCS-Aggres...
Image-Text-to-Text • 35B • Updated Apr 17 • 2.66M • 2.61k
conradlocke/krea2-identity-edit
Updated about 22 hours ago • 154
deepreinforce-ai/Ornith-1.0-35B-GGUF
Text Generation • 35B • Updated 15 days ago • 1.09M • 832
yuxinlu1/gemma-4-12B-agentic-fable5-composer2.5-v2-...
Text Generation • 12B • Updated 21 days ago • 428k • 1.13k
GnLOLot/MiniCPM5-1B-Claude-Opus-Fable5-Thinking-GGUF
Text Generation • 1B • Updated about 20 hours ago • 9.03k • 152
deepseek-ai/DeepSeek-V4-Pro-DSpark
Text Generation • 889B • Updated 6 days ago • 33.1k • 463
open-gigaai/Giga-World-1
Updated 9 days ago • 117

LLM ERA

Zero-Shot Mood Classification

Instead of hand-writing rules, ask a pretrained language model to label text — with zero training examples.

```
from transformers import pipeline

zero_shot = pipeline(
    "zero-shot-classification",
    model="typeform/distilbert-base-uncased-mnli",
    device=-1,  # CPU
)

candidate_moods = ["happy", "sad", "romantic",
    "energetic", "chill", "aggressive"]
zero_shot(track_title, candidate_moods)
```

~250MB distilled model — fast on CPU, no GPU required

Labels never seen in training — defined by YOU at inference time

Great for turning free text into structured categories fast

SECTIONS 11-12

Sentence Embeddings + PCA as New Features

Turn each track title into a dense vector capturing semantic meaning, then compress it into a handful of numeric feature columns.

```
from sentence_transformers import SentenceTransformer
from sklearn.decomposition import PCA

model = SentenceTransformer("all-MiniLM-L6-v2")
embeddings = model.encode(df["track_name"])

pca = PCA(n_components=5, random_state=42)
emb_pca = pca.fit_transform(embeddings)
# emb_pca_1 ... emb_pca_5 -> new feature columns
```

Why compress?

- Raw embeddings: 300-800+ dimensions, too wide for tabular models
- PCA keeps the top signal in ~5 columns
- Downstream K-Means can cluster songs by title 'vibe' with zero manual rules

This is unsupervised learning as a feature source — one of the most powerful ideas in modern feature engineering, and exactly what Question 10 of your assignment asks you to do.

SECTION 13

Putting It All Together: The Final Feature Table

A real pipeline ends here: one wide, clean table ready to hand to next week's ML engineer (also you).

Numeric

`duration_min, loudness_abs, popularity_norm`

Ratio / Interaction

`dance_energy_score, vocal_energy_gap`

Binned / Encoded

`duration_bucket, key_name, mode_name`

Text-based

`num_artists, is_remix, track_word_count`

Aggregation

`genre_avg_popularity, dance_rank_in_genre`

Domain / LLM

`mood_quadrant, zero_shot_mood, emb_pca_1..5`

```
df.to_csv("spotify_features_week2.csv")
print("Final shape:", df.shape)
```

MINDSET

How to Be a Good Data Engineer

01 Understand before you transform

Read .info() / .describe() first. Know units, ranges, and missing values.

02 Prefer readable over clever

A clear def beats a one-line lambda no one can debug at 2am.

03 Interaction > isolation

Ratios and combinations often beat raw columns — think in relationships.

04 Guard against leakage

Ask: could this feature see the future? Group stats are the classic trap.

05 Domain knowledge is a feature

Music theory, business rules, physics — encode what experts already know.

06 Ship a clean, documented table

Your output is someone else's input. Name columns clearly, save your work.

RECAP

What We Covered & What's Next

✅ Today's toolkit

- Numeric transforms, ratios & interactions
- Binning and categorical decoding
- Text-based features from raw strings
- Group aggregation (and its leakage risk)
- Domain-driven mood quadrants
- Zero-shot labels & embeddings (LLM-era FE)

🚀 Week 3 — Next

Supervised Learning

Every feature you engineered this week becomes the input your models next week will actually learn from. Bad features next week = bad models, no matter the algorithm.

Homework preview

Apply the exact same toolkit — numeric, ratio, text, aggregation, and domain features — to a Netflix dataset. Same thinking, new domain.

ASSIGNMENT

10 Feature Engineering Challenges

Same dataset, same toolkit — now you build the columns yourself. Difficulty climbs from a single comparison to unsupervised clustering.

Q1-Q2 ★	Boolean flags & column subtraction (is_short_song, energy_valence_diff)
Q3-Q4 ★★	Multi-condition logic & quantile binning (loud_and_fast, popularity_quartile)
Q5-Q6 ★★★	Regex text features & frequency encoding (title_has_number, artist_track_count)
Q7-Q8 ★★★★	Cross-group ranking & per-genre IQR outlier detection
Q9 ★★★★	Combined categorical feature + frequency encoding
Q10 ★★★★★	K-Means clustering on title-embedding PCA components

THANK YOU

Questions?

Build one feature you're proud of before next class — bring it to Week 3.

Teerapong Panboonyuen (P'Kao) · teerapong.pa@chula.ac.th

github.com/kaopanboonyuen/CP020003_ArtificialIntelligence_2026s1